



ProjectManager

System Design Document

Full-stack collaborative Kanban project management application

Version 1.0

31 July 2026

Repository: github.com/niyomudendezo/projectmanager

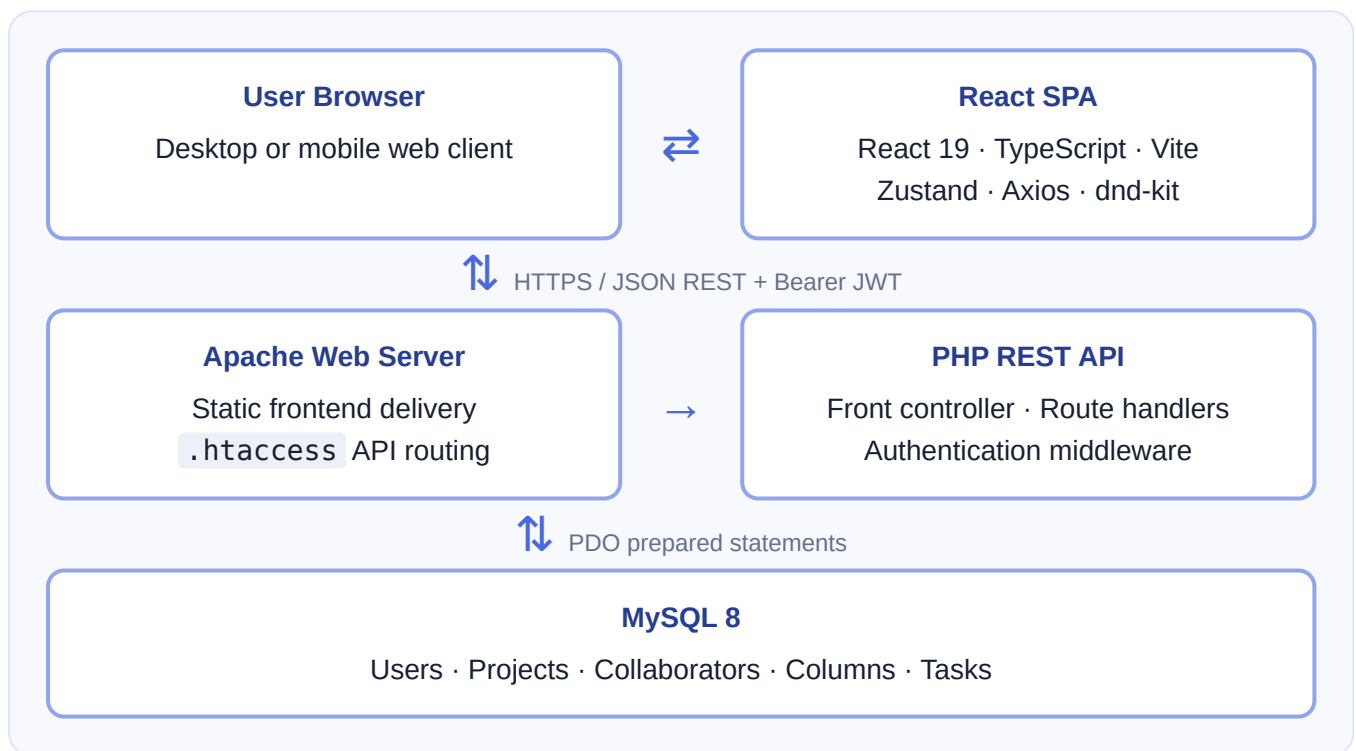
1. Purpose and Scope

ProjectManager lets registered users create projects, organize work on Kanban boards, track priority and schedules, and invite registered collaborators. This document describes the implemented architecture, data model, interfaces, security boundaries, and deployment model.

Primary capabilities

- Account registration and JWT-based sign-in.
- Project creation, editing, deletion, dashboard statistics, and progress.
- Configurable board columns and drag-and-drop task movement.
- Task metadata: description, priority, dates, times, and position.
- Project invitations with pending and accepted states.

2. Architecture Overview



The system uses a three-tier client/server architecture. The browser renders the single-page application. Axios calls a PHP JSON API through one front controller. PHP performs authorization and persistence through PDO; MySQL enforces relationships and cascading cleanup.

Design intent: Keep deployment simple for XAMPP/LAMPP while separating presentation, business/API logic, and persistent data.

3. Component Design

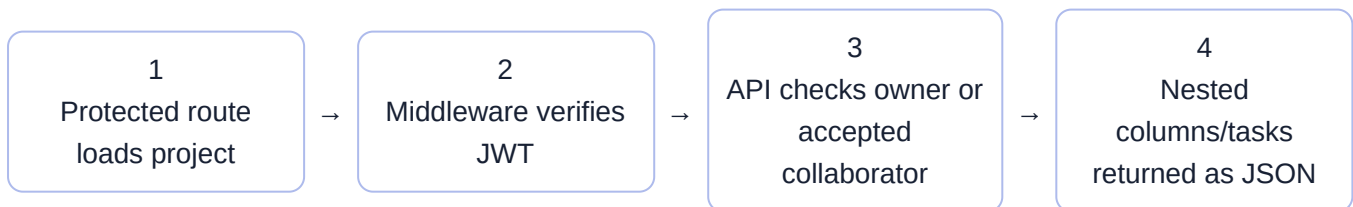
Layer	Components	Responsibilities
Presentation	Pages, Navbar, Sidebar, KanbanBoard, modals, cards	Routing, responsive UI, forms, board interactions, visual task state.
Client state	Zustand authStore and boardStore	Current user/session view, project/column/task state, API actions.
Transport	Axios client	Base API path, JWT injection, centralized 401 logout behavior.
API entry	backend/index.php	CORS, JSON responses, request parsing, resource dispatch, error boundary.
Domain routes	auth, projects, columns, tasks, invitations	Validation, authorization, CRUD, task movement, collaboration workflow.
Security	JWT helpers and auth middleware	Token creation/validation and authenticated user identity.
Persistence	PDO database connector and MySQL schema	Prepared queries, constraints, ordering, relational integrity.

4. Main Request Flows

Authentication



Load and update a board

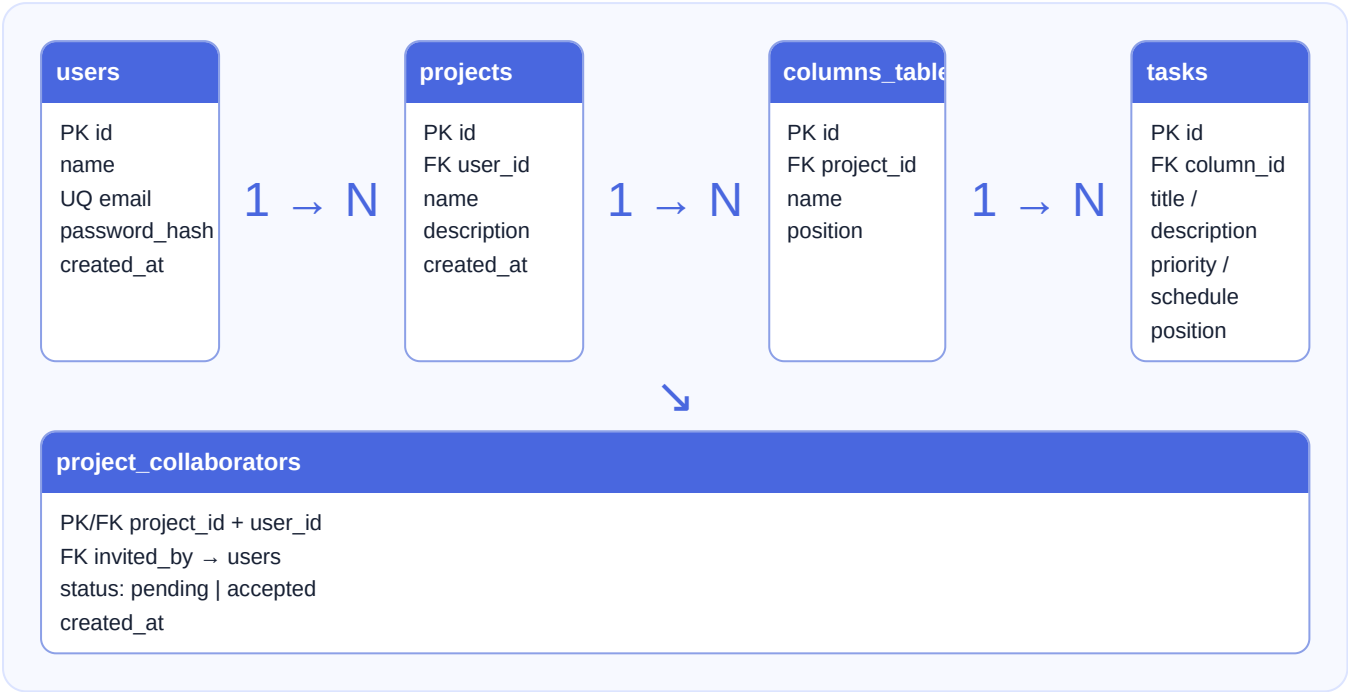


When a task is dragged, the client sends the destination column and position to the task move endpoint. The persisted order becomes the source of truth and the board state is refreshed or reconciled.

Collaboration

Only a project owner can invite a registered user. The join table records the inviter and a `pending` status. After acceptance, the collaborator can load the project; owner-only operations remain restricted.

5. Data Model



Users own many projects. Projects contain ordered columns, and columns contain ordered tasks. Users and projects have a many-to-many collaboration relationship with invitation metadata. Foreign keys use `ON DELETE CASCADE`, so deleting an owner/project/column removes dependent records consistently.

6. API Surface

Resource	Representative operations	Access
Auth	POST /auth/register, POST /auth/login	Public
Projects	List/create; get/update/delete by ID; manage collaborators	Authenticated; ownership rules vary by operation
Columns	Create, rename, delete	Authenticated and project-scoped
Tasks	Create, update, delete, move	Authenticated and project-scoped
Invitations	List invitations; accept an invitation	Authenticated invitee

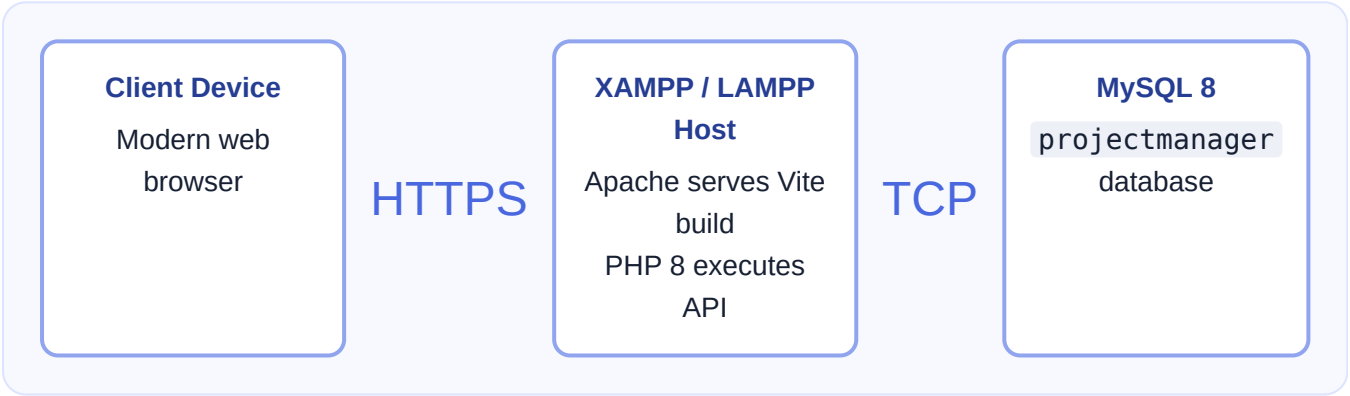
Responses use JSON and standard HTTP status codes including 201, 204, 400, 401, 403, 404, 409, and 500.

7. Security and Reliability

- Passwords are hashed using PHP's bcrypt-backed password API and are never returned.
- Protected endpoints require a signed JWT in the Authorization header.
- Resource queries enforce owner/collaborator scope; collaborator management and project mutation include owner checks.
- PDO prepared statements reduce SQL-injection risk, and input checks reject invalid or missing fields.
- Database foreign keys preserve referential integrity and cascades prevent orphan records.
- The API has a common exception boundary that emits a JSON server-error response.

Production hardening: Serve only over HTTPS; load database credentials and a strong JWT secret from environment configuration; restrict CORS to the deployed frontend origin; avoid exposing exception details; add token expiry/refresh or secure HttpOnly cookies; implement rate limiting, audit logs, automated backups, and CSRF protection if cookie-based sessions are adopted.

8. Deployment



The current target is a single-host deployment suited to development, demonstration, and small workloads. Frontend assets are compiled with Vite and served by Apache. The PHP API and MySQL can run on the same host. A production evolution can place the database on a managed service, add a reverse proxy/TLS endpoint, and deploy multiple stateless API instances because session identity is carried by JWT.

9. Quality Attributes and Trade-offs

Attribute	Current approach	Future direction
Maintainability	Routes separated by domain; typed component/state layer	Service/repository layers and automated API tests
Performance	Simple direct queries; nested board response minimizes client round trips	Indexes, pagination, query consolidation, caching
Scalability	Stateless JWT API; simple single-host deployment	Horizontal API scaling and managed database
Availability	Database constraints and common error handling	Health checks, monitoring, backups, replication

Security	bcrypt, JWT, prepared statements, resource checks	Secret management, expiry/refresh, strict CORS, rate limiting
----------	---	---